

ECafe layihəsində Login, Register, Session Revoke və Refresh Token məntiqi

Bu sənəd backend və frontend-də hazırda tətbiq olunmuş auth/session davranışını sadə dillə izah edir. Məqsəd həm developer, həm də sistemi test edən şəxsin nəyi necə yoxlamalı olduğunu aydın görməsidir.

Qısa nəticə

Sistemdə access token qısa müddətli giriş açarındır, refresh token isə HttpOnly cookie-də saxlanan sessiya açarındır. Refresh token DB-də hash formada saxlanır və rotate olunur. Session revoke əlavə olunduğu üçün tək cihazı bağlamaq və həmin access token-i növbəti API request-də bloklamaq mümkündür.

Əsas anlayışlar

Termin	Layihədə mənası
Access token	Frontend-in API çağırışlarında istifadə etdiyi JWT. İçində userId, role, permissions, sessionVersion və sessionId claim-ləri var.
Refresh token	Uzunömürlü sessiya açarındır. Client-ə response body-də qaytarılır, HttpOnly cookie kimi yazılır.
SessionId	Eyni cihaz/browser sessiyasını tanıdan dəyərdir. Refresh token rotate olsa da eyni sessionId saxlanılır.
SessionVersion	User deaktiv, logout-all və rol dəyişiklikləri kimi hallarda bütün köhnə access token-ləri invalid etmək üçün istifadə olunur.
Redis/cache	Hər request-də DB yükünü azaltmaq üçün user aktivliyi və session aktivliyi cache-də saxlanılır.

Endpoint xəritəsi

Endpoint	İşləmə məqsədi
POST /api/v1/user/login	Email/parol yoxlanılır, yeni sessionId yaradılır, refresh token cookie-yə yazılır, access token qaytarılır.
POST /api/v1/user/register	Yeni user yaradılır, login kimi access token və refresh cookie verilir.
POST /api/v1/user/refresh	Cookie-dəki refresh token yoxlanılır, köhnə token revoke edilir, yeni refresh token yazılır, yeni access token qaytarılır.
POST /api/v1/user/logout	Cari browserdəki refresh token revoke edilir və refresh cookie silinir.
POST /api/v1/user/logout-all	İstifadəçinin bütün refresh token-ləri revoke edilir, sessionVersion artırılır və cache invalid edilir.
GET /api/v1/user/sessions	Profil səhifəsində aktiv sessiyaları göstərmək üçün session siyahısını qaytarır.
DELETE /api/v1/user/sessions/{sessionId}	Seçilmiş sessiyanın aktiv refresh token-lərini revoke edir və session cache-ni silir.

Login flow

POST /api/v1/user/login

1. Email normalize edilir: trim + lowercase.
2. Login attempt lockout yoxlanır.
3. User email ilə tapılır.
4. Şifrə təyin olunubmu və BCrypt ilə uyğundurmu yoxlanılır.
5. User aktiv deyilsə giriş bloklanır.
6. Yeni sessionId yaradılır.
7. Refresh token random yaradılır, hash-lənib DB-yə yazılır.
8. Access token yaradılır və içində sessionId claim olur.
9. Refresh token HttpOnly cookie-yə yazılır, frontend-ə yalnız access token qaytarılır.

Təhlükəsizlik səbəbi

- Refresh token JavaScript ilə oxunmur, çünki HttpOnly cookie-dədir.
- DB-də refresh token-in özü yox, hash-i saxlanılır.
- Login endpoint rate limit altındadır.
- Şifrə BCrypt ilə yoxlanılır, plaintext saxlanmır.

Register flow

- Email və telefon təkrar istifadə olunmamalıdır.
- Profil şəkli varsa attach edilə bilən file kimi yoxlanılır.
- User yaradılır, PasswordSetAt yazılır.
- Yeni sessionId və refresh token yaradılır.
- Login-dəki kimi cookie + access token flow-u işləyir.

Refresh token rotation

POST /api/v1/user/refresh

1. Refresh token request body-dən və ya cookie-dən oxunur.
2. Token SHA-256 hash edilir və DB-də axtarılır.
3. Token tapılmasa, revoke edilibsə və ya expire olubsa request rədd edilir.
4. Əgər revoke olunmuş token yenidən istifadə olunursa, refresh token reuse kimi kritik hadisə sayılır.

5. Köhnə refresh token revoke edilir və ReplacedByTokenHash yazılır.
6. Yeni refresh token yaradılır, amma eyni sessionId saxlanılır.
7. Yeni access token eyni sessionId ilə qaytarılır.

Niyə rotation var?

- Refresh token oğurlansa, köhnə token-in təkrar istifadəsi aşkarlana bilər.
- Reuse aşkarlananda həmin sessiya və ya bütün sessiyalar bağlana bilər.
- Sessiya cihaz səviyyəsində izlənilir, token səviyyəsində qarışıqlıq yaranmır.

Logout və logout-all fərqi

Əməliyyat	Nə edir	Nəticə
Logout	Cari refresh token hash ilə tapılır və revoke edilir. Cookie silinir.	Cari browser çıxış edir. Eyni session növbəti request-də də bloklanır.
Logout all	Bütün aktiv refresh token-lər revoke edilir və user.SessionVersion artırılır.	Bütün cihazlardakı access token-lər növbəti request-də invalid olur.
Single session revoke	Verilmiş sessionId üzrə yalnız həmin user-in aktiv refresh token-ləri revoke edilir.	Seçilmiş cihaz bağlanır, digər cihazlar işləməyə davam edir.

Session revoke flow

DELETE /api/v1/user/sessions/{sessionId}

1. Endpoint authenticated user tələb edir.
2. sessionId boşdursa business error qaytarılır.
3. Query yalnız currentUserId + sessionId üçün işləyir.
4. Tapılan aktiv refresh token-lər revoke edilir.
5. user-session-active:{userId}:{sessionId} cache açarı silinir.
6. Həmin sessiyadan gələn növbəti API request SessionInvalid alır.

Burada vacib security detail

İstifadəçi başqa user-in sessionId dəyərini bilsə belə, query currentUserId ilə filterləndiyi üçün yad sessiyanı bağlaya bilmir.

ActiveUserMiddleware nə edir?

- Authenticated request-lərdə userId oxuyur.
- Access token içində sessionVersion yoxdursa request-i rədd edir.
- Redis/cache üzərindən user aktivliyini və sessionVersion dəyərini yoxlayır.
- sessionVersion fərqlidirsə access token invalid sayılır.
- Access token-də sessionId varsa, həmin session-un aktiv olub-olmadığını yoxlayır.
- Refresh endpoint bu middleware yoxlamasından çıxarılıb ki, token refresh flow-u ayrıca işləyə bilsin.

Cache məntiqi

Cache key	Nə saxlayır	Nə vaxt silinir
user-session-state:{userId}	User aktivliyi və sessionVersion	User deaktiv/logout-all/rol dəyişiklikləri zamanı

user-session-active:{userId}:{sessionId}	Konkret session aktivdirmi	Logout və single session revoke zamanı
--	----------------------------	---

Cache oxunmasa və ya yazılmasa, sistem DB-yə fallback edir. Bu, Redis problemi olanda login/session təhlükəsizliyinin tam dayanmasının qarşısını alır.

Frontend davranışı

- Frontend access token-i saxlayır və API çağırışlarında istifadə edir.
- Refresh token frontend tərəfindən oxunmur; cookie avtomatik backend-ə gedir.
- Profil səhifəsində GET /api/v1/user/sessions çağırılır.
- Cari sessiya badge ilə göstərilir.
- Cari sessiya bağlananda frontend mövcud logout() flow-undan istifadə edir ki, cookie də silinsin.
- Başqa sessiya bağlananda DELETE /api/v1/user/sessions/{sessionId} çağırılır və list yenilənir.
- Logout-all etdikdə frontend auth token-ləri təmizləyir və login səhifəsinə yönləndirir.

Swagger ilə yoxlama checklist-i

Test	Addımlar	Gözlənilən nəticə
Login	POST /user/login ilə aktiv user daxil olsun.	Access token response-da gəlsin, refresh cookie yazılsın.
Refresh	POST /user/refresh çağır.	Yeni access token gəlsin, refresh cookie yenilənsin.
Session list	GET /user/sessions çağır.	Ən azı bir session görünsün və biri isCurrent=true olsun.
İkinci cihaz	Incognito və ya başqa browser ilə login ol.	GET /user/sessions iki session qaytarsın.
Single revoke	Bir sessionId üçün DELETE /user/sessions/{sessionId} çağır.	Həmin session listdən çıxsın və o cihaz növbəti request-də bloklansın.
Logout	POST /user/logout çağır.	Cari refresh token revoke olsun, cookie silinsin.
Logout all	POST /user/logout-all çağır.	Bütün sessiyalar bağlansın, köhnə access token-lər SessionInvalid alsın.

Qeyd və limitlər

- Yeni migration lazım deyil, çünki SessionId artıq UserRefreshToken entity-sində var.
- Device label config-dən idarə olunur, kodda browser/platform siyahısı saxlanmır.
- Ən dəqiq cihaz məlumatı üçün gələcəkdə User-Agent parsing kitabxanası və ayrıca UserSession cədvəli əlavə oluna bilər.
- Bu mərhələdə məqsəd təhlükəsiz, az riskli və mövcud arxitektura uyğun session idarəsidir.

Kodda əsas yerlər

- Backend controller: ECafe.Api/Controllers/AuthController.cs və UserSessionController.cs
- Auth logic: ECafe.Application/Services/Auth/Concrete/AuthManager.cs
- Session logic: ECafe.Application/Services/Auth/Concrete/UserSessionManager.cs

- JWT: ECafe.Application/Services/Jwt/Concrete/JwtManager.cs
- Middleware: ECafe.Api/Middlewares/ActiveUserMiddleware.cs
- Cache: ECafe.Infrastructure/Redis/UserSessionStateCache.cs
- Frontend profile: src/pages/customer/ProfilePage.tsx
- Frontend API: src/shared/api/ecafeApi.ts və endpoints.ts